

DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering) Shahbad
Daulatpur, Bawana Road, Delhi-110042

**DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING**



Machine Learning Lab File
CO303

Submitted To:

Assistant Prof. Gull Kaur
Department of Computer
Science and Engineering

Submitted By:

Krish Bansal
24/CS/248

INDEX

SNo.	Aim	Date	Page No.	Signature
1	To implement and evaluate the K-Nearest Neighbors (KNN) classification algorithm on the Iris dataset.			
2	To formulate a machine learning problem, inspect and assess the health of a dataset, separate features and target variables, split the dataset into training and testing sets, and establish a dummy baseline model for evaluating the usefulness of a machine learning			
3	To understand and implement proper data preprocessing using Scikit-learn pipelines, identify data leakage in a machine learning workflow, and compare the performance of a leaky preprocessing pipeline with a correctly implemented leakage-free pipeline.			
4				
5				
6				
7				
8				
9				

10				
11				
12				

EXPERIMENT – 2

AIM:

To demonstrate data leakage caused by incorrect preprocessing and target-based feature engineering on the Titanic dataset, and to compare flawed workflows with a correct leakage-free scikit-learn pipeline using Logistic Regression.

THEORY:

Data Leakage

Data leakage occurs when information from the test set, which should remain unseen during model development, influences the training process. Leakage can happen when data-dependent preprocessing steps such as imputation, scaling, or encoding are fitted on the complete dataset before the train-test split.

A leakage-free workflow first divides the raw observations into training and testing sets. Any preprocessing operation that learns parameters from data must then be fitted only on the training set.

Preprocessing

The Titanic dataset contains 891 passenger records. The target variable is Survived. The model uses seven features:

- Pclass
- Age
- SibSp

- Parch
- Fare
- Sex
- Embarked

High-cardinality features such as Name and Ticket are excluded from the baseline model. Ticket is used later only for an intentionally leaky target-derived feature.

ColumnTransformer and Pipeline

Numerical and categorical features are processed separately. Numerical features are median-imputed and standardized using StandardScaler. Categorical features are filled using the most frequent category and encoded using OneHotEncoder(handle_unknown='ignore'). Logistic Regression is then used as the binary classifier.

Putting preprocessing and the classifier inside a scikit-learn Pipeline ensures that preprocessing is fitted only on the training partition.

Types of Leakage Demonstrated

- Preprocessing the complete dataset before the train-test split.
- Computing imputation values from the complete dataset before splitting.
- Creating a target-derived Ticket_Surv_Rate feature from the complete dataset before splitting.

Evaluation Metrics

Each workflow is evaluated using Accuracy, Precision, Recall, F1-Score, and ROC AUC. Confusion matrices and ROC curves are also generated.

The train-test split uses 70% training data and 30% testing data with random_state=42.

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    roc_curve, ConfusionMatrixDisplay
)

import kagglehub
import os

path = kagglehub.dataset_download('yasserh/titanic-dataset')
df = pd.read_csv(os.path.join(path, 'Titanic-Dataset.csv'))

num_cols = ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
cat_cols = ['Sex', 'Embarked']

X = df[num_cols + cat_cols]
y = df['Survived']

print(f"X shape: {X.shape}")

import matplotlib.pyplot as plt
import seaborn as sns

def evaluate_model(name, y_true, y_pred, y_prob):
    print(f'\n--- {name} ---')
    print(f'Accuracy: {accuracy_score(y_true, y_pred):.4f}')
    print(f'Precision: {precision_score(y_true, y_pred):.4f}')
    print(f'Recall: {recall_score(y_true, y_pred):.4f}')
    print(f'F1-Score: {f1_score(y_true, y_pred):.4f}')
    print(f'ROC AUC: {roc_auc_score(y_true, y_prob):.4f}')

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=['Died', 'Survived']
    )
    disp.plot(ax=axes[0], cmap='Blues', colorbar=False)
    axes[0].set_title(f'{name} - Confusion Matrix')

    fpr, tpr, _ = roc_curve(y_true, y_prob)
    axes[1].plot(
        fpr, tpr,

```

```

label=f'AUC = {roc_auc_score(y_true, y_prob):.4f}'
)
axes[1].plot([0, 1], [0, 1], 'k--', label='Random')
axes[1].set_title(f'{name} - ROC Curve')
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].legend()

plt.tight_layout()
plt.show()

def get_preprocessor():
    num_pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    cat_pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])

    return ColumnTransformer([
        ('num', num_pipe, num_cols),
        ('cat', cat_pipe, cat_cols)
    ])

# WRONG: preprocessing whole dataset
X_leak = get_preprocessor().fit_transform(X)

X_train_L, X_test_L, y_train_L, y_test_L = train_test_split(
    X_leak, y, test_size=0.3, random_state=42
)

model_leak = LogisticRegression()
model_leak.fit(X_train_L, y_train_L)

preds_leak = model_leak.predict(X_test_L)

evaluate_model(
    'Leaky Pipeline (Preprocessing)',
    y_test_L,
    preds_leak,

```

```

        model_leak.predict_proba(X_test_L)[:, 1]
    )

# CORRECT: split first, then preprocess
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

pipe = Pipeline([
    ('prep', get_preprocessor()),
    ('model', LogisticRegression())
])

pipe.fit(X_train, y_train)
preds_correct = pipe.predict(X_test)

evaluate_model(
    'Correct Pipeline',
    y_test,
    preds_correct,
    pipe.predict_proba(X_test)[:, 1]
)

print(f"Leaky Age Mean (Full Data): {X['Age'].mean():.2f}")
print(f"Correct Age Mean (Train Only): {X_train['Age'].mean():.2f}")

# FLAW 2: Imputing on full data before split
X_imp = X.fillna(X.median(numeric_only=True))

for col in cat_cols:
    X_imp[col] = X_imp[col].fillna(X_imp[col].mode()[0])

X_tr_imp, X_te_imp, y_tr_imp, y_te_imp = train_test_split(
    X_imp, y, test_size=0.3, random_state=42
)

pipe_imp = Pipeline([
    ('prep', ColumnTransformer([
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
    ])),
    ('model', LogisticRegression())
])

```

```

pipe_imp.fit(X_tr_imp, y_tr_imp)

evaluate_model(
    'Imputation Leak',
    y_te_imp,
    pipe_imp.predict(X_te_imp),
    pipe_imp.predict_proba(X_te_imp)[: , 1]
)

# FLAW 3: Target Leakage
X_tgt_leak = X.copy()

X_tgt_leak['Ticket_Surv_Rate'] = (
    df.groupby('Ticket')['Survived'].transform('mean')
)

X_tr_tl, X_te_tl, y_tr_tl, y_te_tl = train_test_split(
    X_tgt_leak, y, test_size=0.3, random_state=42
)

pipe_tl = Pipeline([
    ('prep', ColumnTransformer([
        ('num',
         Pipeline([
             ('imp', SimpleImputer(strategy='median')),
             ('scl', StandardScaler())
         ]),
         num_cols + ['Ticket_Surv_Rate']),
        ('cat',
         Pipeline([
             ('imp', SimpleImputer(strategy='most_frequent')),
             ('ohe', OneHotEncoder(handle_unknown='ignore'))
         ]),
         cat_cols)
    ]),
    ('model', LogisticRegression())
])

pipe_tl.fit(X_tr_tl, y_tr_tl)

evaluate_model(
    'Target Leak',
    y_te_tl,

```



```

pipe_t1.predict(X_te_t1),
pipe_t1.predict_proba(X_te_t1)[: , 1]
)

```

OUTPUT:

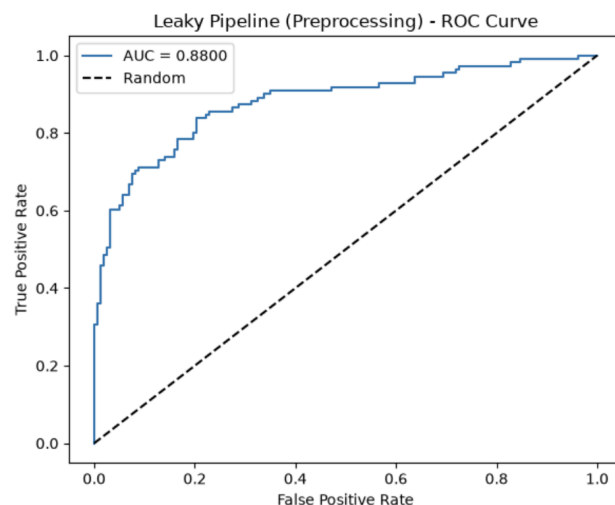
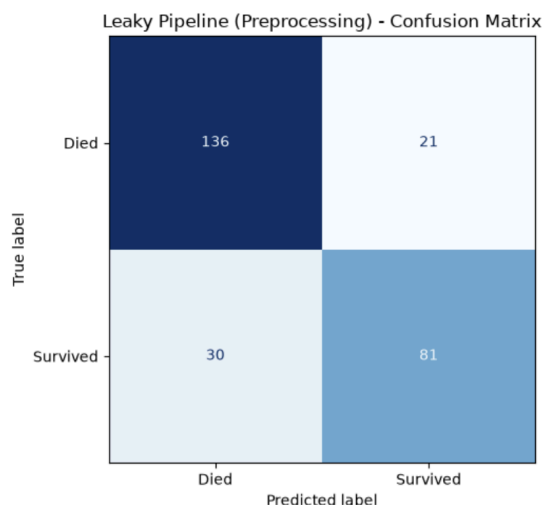
The dataset contains 891 passenger records and the model uses seven features, giving X a shape of (891, 7).

1. Leaky Pipeline – Preprocessing Before Split

In this workflow, `fit_transform()` is applied to the complete feature matrix before the train-test split. Therefore, the imputer, scaler, and encoder learn from both training and testing observations.

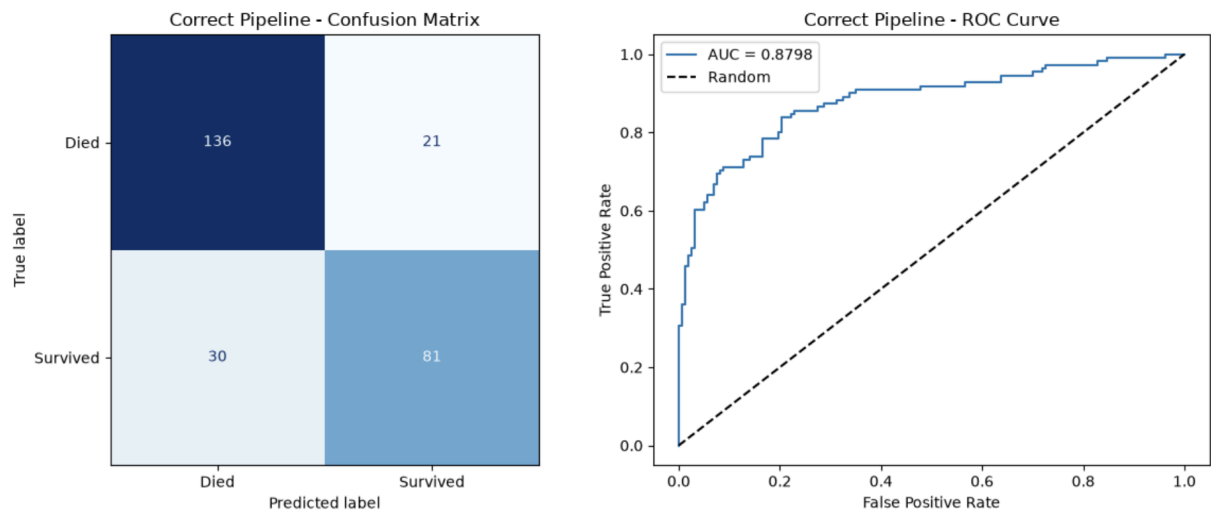
Metric	Value
Accuracy	0.8097
Precision	0.7941
Recall	0.7297
F1-Score	0.7606
ROC AUC	0.8800

Issue: `fit_transform()` on the whole dataset leaks information about the test-data distribution into preprocessing.



2. Correct Leakage-Free Pipeline

The raw data is split first. The preprocessing steps and Logistic Regression model are then placed in a single Pipeline and fitted only on the training partition.



Metric	Value
Accuracy	0.8097
Precision	0.7941
Recall	0.7297
F1-Score	0.7606
ROC AUC	0.8798

3. Comparison of Age Means

Statistic	Value
Leaky Age Mean (Full Data)	29.70
Correct Age Mean (Train Only)	29.26

The difference shows that statistics computed from the complete dataset are not identical to statistics computed from the training partition alone.

4. Full-Data Imputation Leakage

Missing numerical and categorical values are filled using statistics calculated from the complete dataset before splitting.

Metric	Value
Accuracy	0.8097

Precision	0.7941
Recall	0.7297
F1-Score	0.7606
ROC AUC	0.8798

5. Target Leakage Using Ticket Survival Rate

The most severe leakage example creates Ticket_Surv_Rate by grouping the complete dataset by Ticket and calculating the mean Survived value before the train-test split. This feature directly contains information derived from the target labels, including labels belonging to held-out passengers.

Metric	Value
Accuracy	0.9888
Precision	1.0000
Recall	0.9730
F1-Score	0.9863
ROC AUC	0.9999

The extremely high performance is misleading because the feature has access to target information that would not be available for a genuinely unseen passenger.

Overall Comparison

Workflow	Accuracy	Precision	Recall	F1	ROC AUC
Preprocessing before split	0.8097	0.7941	0.7297	0.7606	0.8800
Correct leakage-free pipeline	0.8097	0.7941	0.7297	0.7606	0.8798
Full-data imputation	0.8097	0.7941	0.7297	0.7606	0.8798
Full-data ticket target encoding	0.9888	1.0000	0.9730	0.9863	0.9999

The preprocessing and imputation leaks do not change the recorded metrics for this particular train-test split. However, leakage is still present because information

from held-out observations influences model development. The ticket-based target feature makes the problem especially visible, producing an accuracy of 98.88% and ROC AUC of 0.9999.

LEARNING OUTCOME:

- Understand what data leakage is and why it makes model evaluation unreliable.
- Understand why the train-test split should be performed before fitting data-dependent preprocessing.
- Use ColumnTransformer to process numerical and categorical features separately.
- Use median imputation and StandardScaler for numerical features.
- Use most-frequent imputation and OneHotEncoder for categorical features.
- Build a leakage-free scikit-learn Pipeline combining preprocessing and Logistic Regression.
- Identify leakage caused by full-dataset imputation.
- Understand severe target leakage caused by creating features from target labels before splitting.
- Evaluate classification models using Accuracy, Precision, Recall, F1-Score, and ROC AUC.
- Understand that similar metrics do not prove that a leaky workflow is valid.
- Recognize that the corrected pipeline provides the trustworthy evaluation because the test data remains unseen during model fitting.